

Revisão VA 2

Faculdade UNIDESC

Programação Para Web II

Profº Ruben Prado

ASP.NET Core MVC

- No desenvolvimento de aplicações web com ASP.NET Core MVC, a separação de responsabilidades é fundamental para a manutenção e escalabilidade do sistema.

Integração no Padrão MVC

01

Model: Responsável pelos dados e regras de negócio.

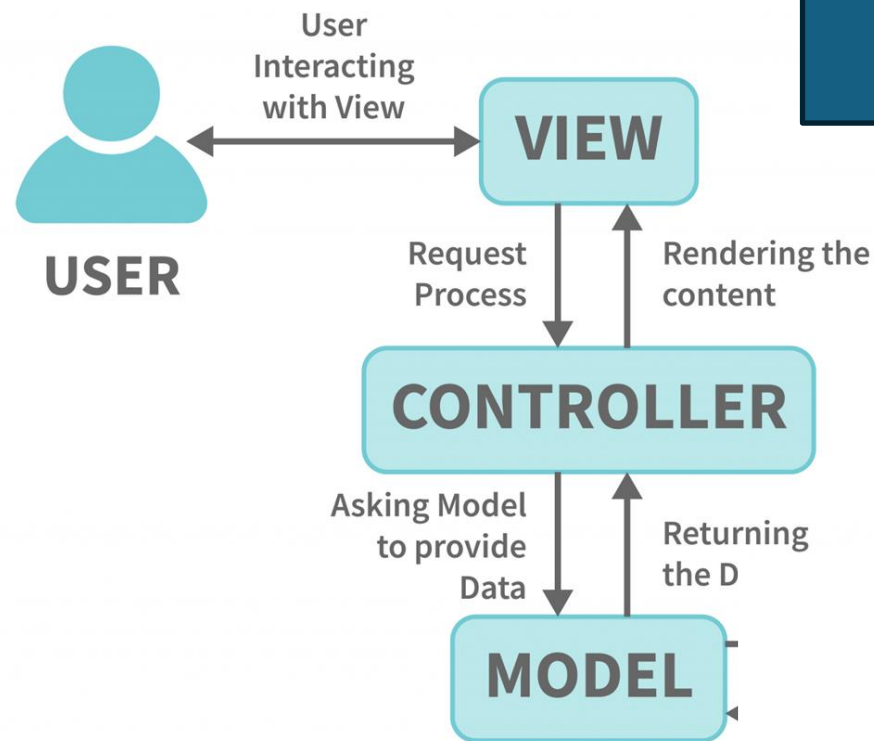
02

View: Responsável pela apresentação.

03

Controller: Coordena a interação entre o Model e a View, manipulando as requisições e definindo a resposta.

MVC



O Padrão MVC divide a aplicação em Model, View e Controller, promovendo a separação de responsabilidades.

A divisão facilita a manutenção do código, pois cada componente tem uma função específica.



```
using UniLanches.Context;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddDbContext<AppDbContext>(option =>
    option.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

Middleware

- São componentes que processam requisições.
- A ordem de registro dos middlewares do pipeline afeta diretamente o comportamento da aplicação;
- Podem ser utilizados para autenticação, logging, tratamento de erros, etc.
- [ASP.NET Core Middleware | Microsoft Learn](#)

AddScoped()

É utilizado para definir o tempo de vida da aplicação.

Quando uma requisição é feita, a instância criada é utilizada pelas dependências;

Cria uma nova instância a cada requisição;

Outros tipos de instâncias:

- `AddTransient()`: Cria uma nova instância do serviço toda vez que ele é solicitado. Ideal para serviços leves e sem estado.
- `AddSingleton()`: Cria uma única instância do serviço para toda a aplicação. Essa instância é compartilhada entre todas as requisições e componentes.

UseRouting()

Controla o roteamento da aplicação. Identificando o endpoint apropriado.

UseEndpoints() e MapControllerRoute()

Utilizado após o UseRouting()

Responsável por executar o endpoint selecionado no roteamento.

Dentro de UseEndpoints(), você define os mapeamentos de rotas para os diversos tipos de endpoints da aplicação.

O método MapControllerRoute() é utilizado dentro de UseEndpoints() para definir rotas convencionais para controllers MVC. Ele estabelece um padrão de URL que mapeia para um controller e uma action específicos.

Entity Framework

ORM – Object Relational Mapper

Mapeia as classes (Models) em tabelas no banco de dados SQL

O Uso do EF permite uma abordagem Code-First

Ainda assim, é necessário o conhecimento de SQL

`dotnet ef migrations add Inicial` – Prepara a Migração

`dotnet ef database update` – Atualiza o BD com a Migração

Como Utilizar o Entity Framework

- Instalar os pacotes necessários
- Configurar o DbContext
- Configurar a string de conexão
- Registrar o DbContext
- Criação a Migration
- Aplicar a Migration

Classes

EF Core

BD

As Classes de Modelo de Domínio (Model)

O Entity Framework Core mapeia os modelos

O Banco de Dados, através das Migrações, são criados e estruturados de acordo com as configurações dos Modelos, incluindo relacionamentos



Views

Views são responsáveis pela a apresentação de dados.

Renderizam a interface do usuário com base em dados fornecidos pela Controller.

Utilizam-se HTML, CSS e JavaScript para melhorar essas interfaces.

- HTML – Estrutura o conteúdo
- CSS – Estilo Visual
- JavaScript - Interatividade

Controller

Processa as requisições do usuário

Possui métodos que recuperam os dados do Banco de Dados